

Essential Terminal Commands

Everything you need to navigate your filesystem and manage files from the command line.

NAVIGATION

<code>pwd</code>	Print current directory
<code>ls</code>	List files in directory
<code>ls -la</code>	List all files with details
<code>cd folder</code>	Enter a folder
<code>cd ..</code>	Go up one level
<code>cd ~</code>	Go to home directory
<code>cd -</code>	Go to previous directory

CREATE & COPY

<code>mkdir name</code>	Create a folder
<code>mkdir -p a/b/c</code>	Create nested folders
<code>touch file.txt</code>	Create an empty file
<code>cp src dest</code>	Copy a file
<code>cp -r dir/ dest/</code>	Copy a directory

MOVE & DELETE

<code>mv old new</code>	Move or rename
<code>rm file</code>	Delete a file
<code>rm -rf dir/</code>	Delete a directory

VIEW & SEARCH

<code>cat file</code>	Print file contents
<code>head -20 file</code>	First 20 lines
<code>tail -20 file</code>	Last 20 lines
<code>grep "text" file</code>	Search in file
<code>grep -r "text" .</code>	Search recursively
<code>find . -name "*.ts"</code>	Find files by pattern

Pro tip: Press `Tab` to auto-complete file and folder names. Press `Ctrl+R` to search your command history.

Claude Code Prompt Patterns

Write better prompts to get better code. These patterns work in Claude Code and any AI coding tool.

EFFECTIVE PROMPTS

- Be specific: "Add a login form with email and password fields" not "add auth"
- Provide context: "In the Express server at server.ts, add a GET /api/users endpoint"
- State constraints: "Use only standard library, no external packages"
- Ask for explanations: "Explain each step as you implement it"

COMMON PROMPT TEMPLATES

- **New feature:** "Create a [component] that [does X]. It should [constraints]."
- **Bug fix:** "The [feature] is [broken behavior]. Expected: [correct behavior]. Fix it."
- **Refactor:** "Refactor [file] to [pattern]. Keep the same behavior."
- **Tests:** "Write tests for [function] covering [edge cases]."

CLAUDE CODE SLASH COMMANDS

<code>/init</code>	Create a CLAUDE.md for your project
<code>/compact</code>	Summarize conversation to save context
<code>/clear</code>	Reset conversation
<code>/cost</code>	Show token usage
<code>/help</code>	Show all commands
<code>/vim</code>	Toggle vim mode

KEYBOARD SHORTCUTS

<code>Esc</code>	Cancel current action
<code>Tab</code>	Accept suggestion
<code>Shift+Tab</code>	Toggle auto-accept mode
<code>Ctrl+C</code>	Stop generation

Pro tip: Start your project with `claude /init` to generate a CLAUDE.md. This file teaches Claude about your project's conventions, tech stack, and patterns.

Git Quick Reference

Version control essentials. Every command you need for daily Git workflow.

SETUP & INIT

<code>git init</code>	Initialize a new repo
<code>git clone url</code>	Clone a remote repo
<code>git status</code>	Show working tree status
<code>git log --oneline</code>	Compact commit history

REMOTE

<code>git push</code>	Push to remote
<code>git pull</code>	Pull from remote
<code>git push -u origin main</code>	Set upstream & push
<code>git remote -v</code>	Show remotes

STAGE & COMMIT

<code>git add file</code>	Stage a specific file
<code>git add .</code>	Stage all changes
<code>git commit -m "msg"</code>	Commit with message
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes

UNDO & FIX

<code>git restore file</code>	Discard changes
<code>git restore --staged f</code>	Unstage a file
<code>git stash</code>	Stash changes
<code>git stash pop</code>	Restore stashed changes
<code>git reset HEAD~1</code>	Undo last commit (keep files)

BRANCHES

<code>git branch</code>	List branches
<code>git branch name</code>	Create new branch
<code>git checkout name</code>	Switch to branch
<code>git checkout -b name</code>	Create & switch
<code>git merge name</code>	Merge branch into current

Pro tip: Use `git commit -m "feat: description"` with conventional commit prefixes: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`

CLAUDE.md & MCP Quick Start

Configure Claude Code for your project and extend it with MCP servers.

CLAUDE.MD STARTER TEMPLATE

```
# Project Name

## Tech Stack
- Language: TypeScript
- Framework: Next.js 16
- Database: Supabase (PostgreSQL)
- Styling: Tailwind CSS 4

## Conventions
- Use functional components with hooks
- Prefer named exports
- Tests live next to source files

## Commands
- `npm run dev` — start dev server
- `npm test` — run tests
- `npm run build` — production build
```

WHERE CLAUDE.MD FILES GO

~/CLAUDE.md	Global (all projects)
./CLAUDE.md	Project root (shared)
./.claude/CLAUDE.md	Personal (gitignored)
./src/CLAUDE.md	Directory-specific

MCP SERVER BASICS

- **What:** Plugins that give Claude new tools (database access, APIs, etc.)
- **Where:** Configure in `.claude/settings.json`
- **How:** `claude mcp add name command`

COMMON MCP SERVERS

filesystem	Read/write files outside project
postgres	Query databases directly
github	PR reviews, issues, actions
playwright	Browser automation & testing

Keep learning: This cheat sheet covers Weeks 1–3 of the free 12-week course. Visit agentcodeacademy.com for the full curriculum including databases, APIs, authentication, testing, and advanced MCP server development.